# 🔹 1. Event Loop

### 🔍 Summary:

The **Event Loop** in Node.js is the **mechanism** that handles **asynchronous operations** (like I/O tasks, timers, promises) in a **single-threaded** environment.

It allows Node.js to be **non-blocking** and handle **many requests at once** without multithreading.

---

### 👨‍💼 Interview Q&A:

**Q1: What is the Event Loop in Node.js? A1:** The event loop is what allows Node.js to perform non-blocking I/O operations by offloading tasks to the system and handling callbacks in a single thread.

**Q2: Which phases exist in the Event Loop? A2:** Major phases are:

- Timers (setTimeout, setInterval)
- I/O Callbacks
- Idle, prepare
- Poll
- Check (setImmediate)
- Close callbacks

**Q3: How does Node.js handle async code with a single thread? A3:** Heavy I/O tasks are delegated to the OS or thread pool (libuv), and their completion is managed via callbacks queued into the event loop.

**Q4: What is the difference between `process.nextTick()` and `setImmediate()`? A4:** `process.nextTick()` schedules a callback **before** the next event loop phase. `setImmediate()` runs **after** the current poll phase.

---

# 🔹 2. Streams

### 🔍 Summary:

**Streams** in Node.js are **interfaces** for **reading or writing** data piece-by-piece instead of loading it all at once (useful for files, network, video).

There are 4 types: **Readable**, **Writable**, **Duplex**, and **Transform**.

---

### 👨‍💼 Interview Q&A:

**Q1: What is a Stream in Node.js? A1:** A stream is a continuous flow of data that can be read/written without loading the entire dataset into memory.

**Q2: Types of Streams? A2:**

- Readable (e.g., `fs.createReadStream`)
- Writable (e.g., `fs.createWriteStream`)
- Duplex (read and write both, e.g., sockets)
- Transform (modify data while reading/writing, e.g., zlib compression)

**Q3: How to listen to data from a Readable Stream? A3:**

```
stream.on('data', (chunk) => { console.log(chunk); });
stream.on('end', () => { console.log('Done reading.'); });
```

**Q4: What is backpressure in Streams? A4:** Backpressure happens when the readable stream pushes data faster than the writable stream can handle, requiring buffering.

---

## 🔹 3. Modules

### 🔍 Summary:

**Modules** in Node.js are reusable blocks of code organized into files. Node.js uses **CommonJS** (`require`) and also supports **ES Modules** (`import/export`).

Built-in modules: `fs`, `http`, `path`, etc.

---

### 👨‍💼 Interview Q&A:

**Q1: How do modules work in Node.js? A1:** Each file is treated as a separate module. You can export parts of a file (functions, objects) and import them into others.

**Q2: What is CommonJS? A2:** CommonJS is the module system used by Node.js where you use `require()` to import and `module.exports` to export.

**Q3: Difference between `require` and `import`? A3:**

- `require()` is synchronous (CommonJS)
- `import` / `export` is asynchronous and part of ES6 Modules (needs `"type": "module"` in `package.json`).

**Q4: How to create a custom module? A4:**

```
// math.js
function add(a, b) { return a + b; }
module.exports = { add };
```

```
// app.js
const { add } = require('./math');
console.log(add(2, 3));
```

---

# 🔷 4. Express.js

## 🔍 Summary:

**Express.js** is a minimal and flexible **Node.js web framework** for building APIs and web applications. It simplifies server creation, routing, middleware, and handling requests/responses.

---

## 👨‍💼 Interview Q&A:

**Q1: What is Express.js? A1:** Express.js is a lightweight web framework built on top of Node.js that simplifies routing, middleware handling, and HTTP server management.

**Q2: How do you define a route in Express? A2:**

```
app.get('/hello', (req, res) => {
  res.send('Hello World');
});
```

**Q3: What are middleware functions in Express? A3:** Functions that have access to the request, response, and next function. Used for tasks like logging, parsing, authentication.

**Q4: How do you handle 404 errors in Express? A4:**

```
app.use((req, res) => {
  res.status(404).send('Not Found');
});
```

---

# 🔷 5. Middleware (in Express.js)

## 🔍 Summary:

**Middleware** are **functions** that run **during** the request/response lifecycle. They can:

- Modify `req` or `res`
- End the request
- Call the next middleware in chain

**Examples**: Logging, Authentication, Error handling, Parsing Body.

---

## 👨‍💼 Interview Q&A:

**Q1: What is middleware in Express.js? A1:** Middleware functions take `req`, `res`, and `next`. They modify the request/response or end the response, or pass control to the next middleware.

**Q2: What is the signature of a middleware? A2:**

```
function middleware(req, res, next) {
  // logic
  next(); // call next middleware
}
```

**Q3: Example of a simple middleware to log request URL? A3:**

```
app.use((req, res, next) => {
  console.log(`Request URL: ${req.url}`);
```

```
  next();
});
```

**Q4: How do you handle errors with middleware? A4:** Error-handling middleware must have **4 parameters** `(err, req, res, next)`.

```
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).send('Something broke!');
});
```

---

# 🔹 6. REST APIs (in Node.js/Express)

## 🔍 Summary:

**REST APIs** are server-side APIs that follow HTTP principles:

- GET (fetch)
- POST (create)
- PUT/PATCH (update)
- DELETE (remove)

Express makes building REST APIs very simple.

---

## 👨‍💼 Interview Q&A:

**Q1: What is REST API? A1:** A set of endpoints exposing resources over HTTP, using methods like GET, POST, PUT, DELETE.

**Q2: Example of full CRUD API in Express? A2:**

```
const express = require('express');
const app = express();
app.use(express.json());

let users = [];

app.get('/users', (req, res) => res.json(users));
```

```
app.post('/users', (req, res) => {
  users.push(req.body);
  res.status(201).json({ message: 'User created' });
});

app.put('/users/:id', (req, res) => {
  users[req.params.id] = req.body;
  res.json({ message: 'User updated' });
});

app.delete('/users/:id', (req, res) => {
  users.splice(req.params.id, 1);
  res.json({ message: 'User deleted' });
});
```

**Q3: What HTTP status codes are important for REST APIs? A3:**

- 200 OK (Success GET)
- 201 Created (Success POST)
- 400 Bad Request (Invalid data)
- 404 Not Found (Resource missing)
- 500 Internal Server Error (Server issues)

---

# 🔹 7. npm Modules

## 🔍 Summary:

**npm (Node Package Manager)** is the **default package manager** for Node.js. It manages libraries, tools, and dependencies.

- Install packages globally or locally
- Versioning, publishing custom packages

---

## 👨‍💼 Interview Q&A:

**Q1: What is npm? A1:** A package manager that installs libraries, manages dependencies, and allows you to publish your own packages.

**Q2: How to install a local package? A2:**

```
npm install lodash
```

**Q3: How to install a global package? A3:**

```
npm install -g nodemon
```

**Q4: Example of using an installed module ( `lodash` ) A4:**

```
const _ = require('lodash');

let arr = [1, 2, 3, 4];
let reversed = _.reverse(arr);
console.log(reversed); // [4, 3, 2, 1]
```

**Q5: How to initialize your own npm package? A5:**

```
npm init
# Creates package.json
```

---

# 🔹 8. File Upload (Handling uploads with Express)

## 🔍 Summary:

Node.js (via Express + `multer` package) allows **handling file uploads** easily by parsing `multipart/form-data` .

---

## 👨‍💼 Interview Q&A:

**Q1: How to upload a file in Node.js? A1:** Use the `multer` middleware to handle file uploads.

**Q2: Example setup for file upload with multer? A2:**

```
const express = require('express');
const multer  = require('multer');
const upload = multer({ dest: 'uploads/' });

const app = express();
```

```
app.post('/upload', upload.single('avatar'), (req, res) => {
  console.log(req.file); // uploaded file info
  res.send('File uploaded!');
});
```

**Q3: What does `upload.single('fieldname')` mean? A3:** It handles a **single file** uploaded under the field name `'avatar'` in the form.

**Q4: How to handle multiple file uploads? A4:**

```
app.post('/uploads', upload.array('photos', 5), (req, res) => {
  console.log(req.files); // Array of uploaded files
});
```

---

# 🔹 9. WebSocket (Real-time communication)

## 🔍 Summary:

**WebSocket** enables **bi-directional communication** between the client and server. Unlike HTTP, it's persistent and doesn't need to re-establish connection every time.

In Node.js, you commonly use:

- Native `ws` package
- Or integrate with `socket.io` (more features, fallback support)

---

## 👨‍💼 Interview Q&A:

**Q1: What is WebSocket used for? A1:** For real-time apps like chat, notifications, live dashboards, multiplayer games — where the server needs to send data anytime without a new client request.

**Q2: Example using `ws` module:**

```
const WebSocket = require('ws');
const server = new WebSocket.Server({ port: 8080 });

server.on('connection', ws => {
```

```
  ws.send('Hello from server');
  ws.on('message', msg => console.log('Client says:', msg));
});
```

**Q3: What is `socket.io` and why is it popular? A3:** It builds on WebSocket and adds:

- Fallback to HTTP long-polling
- Rooms/channels
- Auto reconnection
- Event-based APIs

**Q4: Basic `socket.io` usage:**

```
const io = require('socket.io')(3000);
io.on('connection', socket => {
  socket.emit('welcome', 'You are connected');
  socket.on('chat', msg => console.log('Chat:', msg));
});
```

---

# 🔷 10. Authentication (Concept & Implementation)

## 🔍 Summary:

Authentication verifies **who the user is**. In Node.js, you commonly use:

- Sessions + cookies (traditional)
- JWT tokens (stateless)
- Passport.js (authentication middleware)

---

## 👨‍💼 Interview Q&A:

**Q1: What are common authentication methods in Node.js? A1:**

- Session-based (with cookies + server-side storage)
- Token-based (JWT stored on client)
- OAuth2, SAML (for third-party login)

**Q2: Example of a simple username/password check:**

```
app.post('/login', (req, res) => {
  const { username, password } = req.body;
  if (username === 'admin' && password === 'secret') {
    res.send('Login success');
  } else {
    res.status(401).send('Invalid');
  }
});
```

**Q3: How do you persist login sessions? A3:** Use `express-session`:

```
const session = require('express-session');
app.use(session({ secret: 'mykey', resave: false, saveUninitialized: true }));
```

**Q4: What's the difference between authentication and authorization? A4:**

- Authentication: verifying identity (login)
- Authorization: access control (roles, permissions)

---

# 🔷 11. CLI Tools

## 🔍 Summary:

Node.js can build **command-line tools** using `process.argv`, `commander`, `yargs`, or `inquirer`.

Useful for internal scripts, scaffolding tools (e.g., `create-react-app`), automation, deployment scripts.

---

## 👨‍💼 Interview Q&A:

**Q1: How to access command-line arguments in Node.js? A1:** Use `process.argv`:

```
// node greet.js John
const name = process.argv[2];
console.log(`Hello, ${name}`);
```

**Q2: Better CLI parsing with `commander`:**

```
const { Command } = require('commander');
const program = new Command();

program.version('1.0.0');
program.option('-n, --name <name>', 'Your name');

program.parse(process.argv);
console.log(`Hello, ${program.opts().name}`);
```

**Q3: What is `inquirer` used for? A3:** To create **interactive CLI** prompts:

```
const inquirer = require('inquirer');

inquirer.prompt([
  { type: 'input', name: 'username', message: 'Enter username:' }
]).then(answers => console.log(answers.username));
```

---

# 🔹 12. JWT (JSON Web Token)

## 🔍 Summary:

**JWT** is a **token format** used to securely transmit user identity and claims. Node.js can generate and verify JWTs using `jsonwebtoken`.

Used in **stateless auth systems** where session data lives on the client.

---

## 👨‍💼 Interview Q&A:

**Q1: What does a JWT contain? A1:** Three parts:

1. Header (algo, type)
2. Payload (user info, claims)
3. Signature (to verify authenticity)

**Q2: How to sign and verify JWT in Node.js?**

```
const jwt = require('jsonwebtoken');

const token = jwt.sign({ userId: 123 }, 'secret', { expiresIn: '1h' });
```

```
const decoded = jwt.verify(token, 'secret');
console.log(decoded.userId); // 123
```

**Q3: Where is JWT typically stored in frontend? A3:** Usually in `localStorage` or `HTTP-only cookies`.

**Q4: How do you protect a route using JWT?**

```
function verifyToken(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
  try {
    req.user = jwt.verify(token, 'secret');
    next();
  } catch {
    res.status(401).send('Unauthorized');
  }
}
```

---

# 🔹 13. MVC Architecture (Model-View-Controller)

## 🔍 Summary:

**MVC** is a design pattern for separating application logic into three layers:

- **Model**: Data and business logic
- **View**: UI rendering (HTML)
- **Controller**: Handles input and binds Model ↔ View

In Node.js, this is widely used with frameworks like **Express.js** and **EJS/Pug**.

---

## 👨‍💼 Interview Q&A:

**Q1: What is MVC in Node.js? A1:** It stands for Model-View-Controller. It separates concerns by dividing app logic into:

- Model: data and DB logic
- View: UI templates
- Controller: user input and route handling

**Q2: Folder structure example?**

```
/controllers/userController.js
/models/userModel.js
/views/user.ejs
/routes/userRoutes.js
/app.js
```

**Q3: Example controller and model:**

```
// userModel.js
exports.getUser = () => ({ name: 'Chirag', age: 28 });

// userController.js
const userModel = require('../models/userModel');
exports.showUser = (req, res) => {
  const user = userModel.getUser();
  res.render('user', { user });
};
```

---

# 🔹 14. Template Engines

## 🔍 Summary:

Template engines like **EJS**, **Pug**, and **Handlebars** help render dynamic HTML from server-side data.

Used in:

- Server-side rendering (SSR)
- Email templates
- Admin panels

---

## 👨‍💼 Interview Q&A:

**Q1: What is a template engine in Node.js? A1:** It's a tool that lets you **embed dynamic data** inside HTML templates using a syntax like `<%= %>` (EJS).

**Q2: Popular template engines? A2:**

- EJS (Embedded JS)
- Pug (indent-based syntax)
- Handlebars (logic-less templates)

**Q3: Example using EJS:**

```
app.set('view engine', 'ejs');

app.get('/', (req, res) => {
  res.render('index', { name: 'Chirag' });
});
```

**index.ejs**

```
<h1>Welcome, <%= name %>!</h1>
```

---

# 🔷 15. Event Emitters (Core Node.js)

## 🔍 Summary:

The **EventEmitter** class in Node.js allows you to create and handle custom events — useful for **decoupled, async event-driven code**.

It powers **many core modules** like `fs`, `http`, `net`, etc.

---

## 👨‍💼 Interview Q&A:

**Q1: What is EventEmitter in Node.js? A1:** A Node.js class that lets you define and listen to custom events, useful in building event-driven systems.

**Q2: Basic usage example:**

```
const EventEmitter = require('events');
const emitter = new EventEmitter();

emitter.on('message', msg => console.log('Received:', msg));
emitter.emit('message', 'Hello world');
```

**Q3: What are `.on()` and `.emit()`? A3:**

- `.on()` registers a listener
- `.emit()` triggers the event

**Q4: Can EventEmitters be extended in classes? A4:**

```
class Logger extends EventEmitter {
  log(msg) {
    this.emit('log', msg);
  }
}
```

---

# 🔹 16. Realtime Apps

## 🔍 Summary:

Realtime apps **instantly reflect data updates** without needing manual refresh — e.g., chat, stock tickers, games.

Built using:

- **WebSocket** (raw or via `socket.io`)
- **Pub/Sub** (Redis, Kafka)
- EventEmitters, DB triggers

---

## 👨‍💼 Interview Q&A:

**Q1: What makes an app real-time? A1:** It updates data in the UI **instantly**, without polling. It uses persistent connections like WebSocket.

**Q2: Real-time example using `socket.io`:**

```
const io = require('socket.io')(3000);

io.on('connection', socket => {
  console.log('User connected');
  socket.on('chat', msg => {
    io.emit('chat', msg); // Broadcast to all clients
```

```
  });
});
```

**Q3: How do you scale real-time apps? A3:** Use **Redis adapter** for socket.io to share messages across instances.

```
const redisAdapter = require('socket.io-redis');
io.adapter(redisAdapter({ host: 'localhost', port: 6379 }));
```

**Q4: What's the difference between polling and real-time? A4:**

- Polling: Repeatedly checks the server every few seconds
- Real-time: Persistent connection, server pushes updates instantly

---

# 🔹 17. Auth Middleware

## 🔍 Summary:

**Auth Middleware** is a reusable function placed in the request chain to **verify authentication** (e.g., JWT tokens, sessions) before reaching protected routes.

Commonly used in Express.js to:

- Check if the user is logged in
- Validate tokens or session IDs
- Reject unauthorized access

---

## 👨‍💼 Interview Q&A:

**Q1: What is authentication middleware in Express.js? A1:** A middleware function that checks credentials (token, session) before allowing access to secure endpoints.

**Q2: JWT Auth Middleware Example:**

```
const jwt = require('jsonwebtoken');

function authMiddleware(req, res, next) {
```

```
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).send('Access denied');

  try {
    req.user = jwt.verify(token, 'secret');
    next();
  } catch {
    res.status(403).send('Invalid token');
  }
}
```

**Q3: How do you use this middleware?**

```
app.get('/dashboard', authMiddleware, (req, res) => {
  res.send(`Welcome user ${req.user.id}`);
});
```

**Q4: What's the difference between auth and access control middleware? A4:**

- Auth middleware verifies **who** the user is.
- Access control checks **what** the user is allowed to do (e.g., roles, permissions).

---

# 🔹 18. EJS Templates

## 🔍 Summary:

**EJS (Embedded JavaScript Templates)** lets you write **server-side HTML** with embedded JavaScript logic. It's one of the most widely used template engines in Express.js.

---

## 👨‍💼 Interview Q&A:

**Q1: What is EJS and how does it work? A1:** EJS allows embedding JS code into HTML using tags like `<%= %>`, `<% %>`, and renders templates with dynamic content.

**Q2: How to use EJS in Express?**

```
app.set('view engine', 'ejs');

app.get('/', (req, res) => {
```

```
  res.render('home', { user: 'Chirag' });
});
```

**home.ejs**

```
<h1>Hello <%= user %>!</h1>
```

**Q3: What is the difference between `<% %>` and `<%= %>`? A3:**

- `<%= %>` outputs HTML-escaped content
- `<% %>` runs JavaScript but doesn't output

---

# 🔷 19. SSR Frameworks (Server-Side Rendering)

## 🔍 Summary:

**Server-Side Rendering (SSR)** is rendering HTML on the server instead of the browser. This improves:

- SEO
- First page load time
- Crawlability

Popular frameworks:

- **Next.js** (React)
- **Nuxt.js** (Vue)
- **Express + EJS/Pug** (classic)

---

## 👨‍💼 Interview Q&A:

**Q1: What is Server-Side Rendering (SSR)? A1:** SSR means generating the full HTML response on the server for each route and sending it to the browser, instead of relying solely on client-side JS.

**Q2: Benefits of SSR? A2:**

- SEO-friendly (Google can index content)
- Faster initial page load
- Better for slow devices/networks

**Q3: Example using Express + EJS as an SSR approach:**

```
app.get('/product/:id', (req, res) => {
  const product = getProduct(req.params.id); // DB call
  res.render('product', { product });
});
```

**Q4: SSR vs SPA? A4:**

- SSR: HTML generated on server (good for SEO, slower dev cycles)
- SPA: HTML generated on client (faster routing, poor SEO unless pre-rendered)

---

# 🔷 20. API Gateway

## 🔍 Summary:

An **API Gateway** is a server that acts as an **entry point** for routing client requests to various backend services (like microservices).

It also handles:

- Authentication
- Rate limiting
- Logging
- Aggregating multiple services into one response

Examples: **Kong**, **Express Gateway**, **AWS API Gateway**, **NGINX**

---

## 👨‍💼 Interview Q&A:

**Q1: What is an API Gateway? A1:** It's a reverse proxy server that routes and manages API calls between clients and multiple backend services.

**Q2: When do you use an API Gateway? A2:**

- When you have **microservices**
- Want to **centralize** rate-limiting, caching, logging, authentication

**Q3: Can you implement a basic gateway in Express?**

```
const express = require('express');
const { createProxyMiddleware } = require('http-proxy-middleware');

const app = express();

app.use('/users', createProxyMiddleware({ target: 'http://localhost:4000', changeOrigin: true }))
app.use('/orders', createProxyMiddleware({ target: 'http://localhost:5000', changeOrigin: true })
```

**Q4: How does API Gateway differ from Load Balancer? A4:**

- API Gateway → Application-level routing, transformations, auth
- Load Balancer → Low-level traffic distribution (L4/L7), mostly for redundancy

---